



ALLIANCE UNIVERSITY

*Private University established in Karnataka State by Act No.34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi*

Mini-Project Report

Bachelor of Computer Applications

Semester IV

Full Stack Development

By

Vishesh Gurnani

2411021240033

Department of Computer Applications

Alliance University

Chandapura-Anekal Main Road, Anekal

Bengaluru 562106

March 2026

Introduction

This project demonstrates the integration of Postman, MongoDB Compass, and Visual Studio Code to build a basic REST API-based student management system. Using Node.js and Express, a web server was created that connects to a MongoDB database and performs CRUD (Create, Read, Update, Delete) operations. Postman was used to test the API endpoints, while MongoDB Compass was used to visually manage and verify the stored data. The system allows student details to be stored and retrieved in JSON format through HTTP requests.



In this project, MongoDB serves as the database for storing student information such as name, age, and course. Mongoose is an Object Data Modelling (ODM) toolkit for defining data structure and effectively interacting with databases. The API endpoints are built in Node.js with Express, which processes HTTP requests from clients. Postman is used to test API requests, and MongoDB Compass offers a graphical user interface for viewing and managing stored data.



This project will allow us to understand how backend services communicate with databases and how APIs are used to transfer data between applications. It also provides practical exposure to technologies and testing them using API tools, this project demonstrates the fundamentals of building and managing a simple backend service.

Objectives

The main objective of this project was to successfully connect **VS Code, MongoDB Compass, and Postman** to create a functioning REST API server. The server should be able to store student details in a MongoDB database and return them in JSON format when requested through API endpoints.

Tools and Technologies Used

- **Postman** – Used for testing REST API endpoints such as POST, GET, PUT, and DELETE requests.
- **MongoDB Compass** – Used as a graphical interface to create and manage the MongoDB database and collections.
- **Visual Studio Code (VS Code)** – Used as the development environment to write and manage the Node.js server code.

Project Setup (Commands Used)

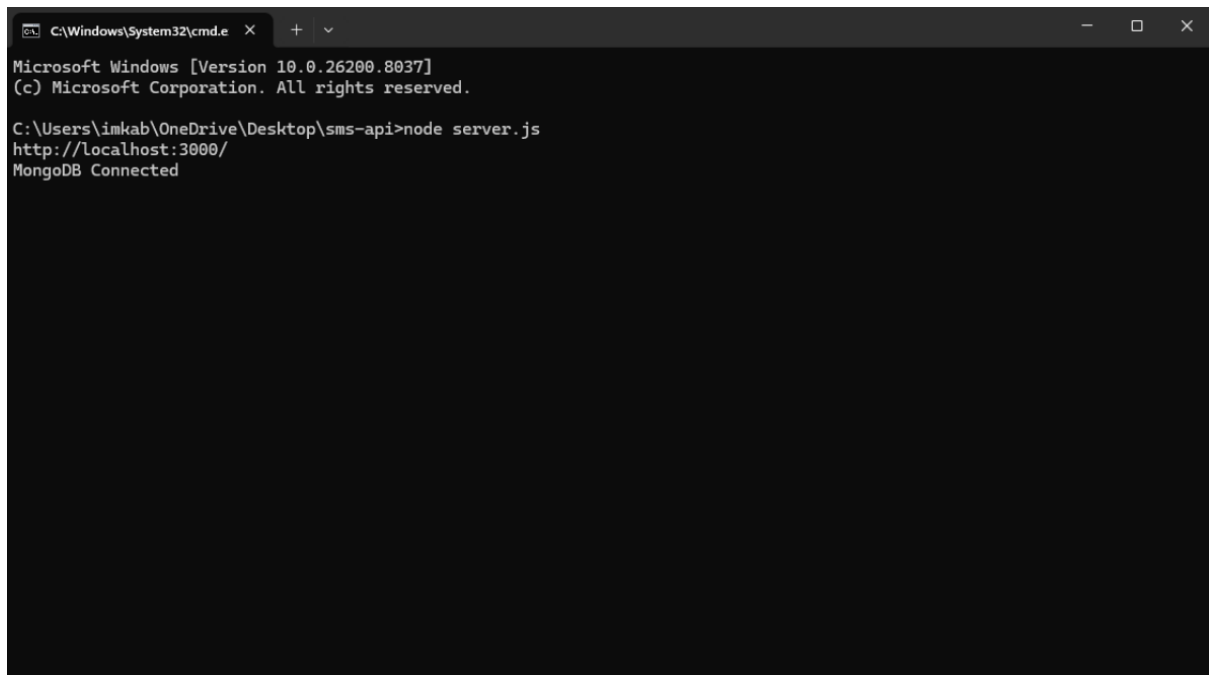
The project server was started using the command below in the terminal.

```
mkdir sms-api  
cd sms-api  
npm init -y  
npm install express mongoose body-parser
```

To run the server:

```
node server.js
```

This command runs the Node.js server and connects the application code written in VS Code with the MongoDB database.



```
C:\Windows\System32\cmd.e  x  +  v  
Microsoft Windows [Version 10.0.26200.8037]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\imkab\OneDrive\Desktop\sms-api>node server.js  
http://localhost:3000/  
MongoDB Connected
```

MongoDB Database Setup

Default Connection URL: `mongodb://127.0.0.1:27017/studentdb`

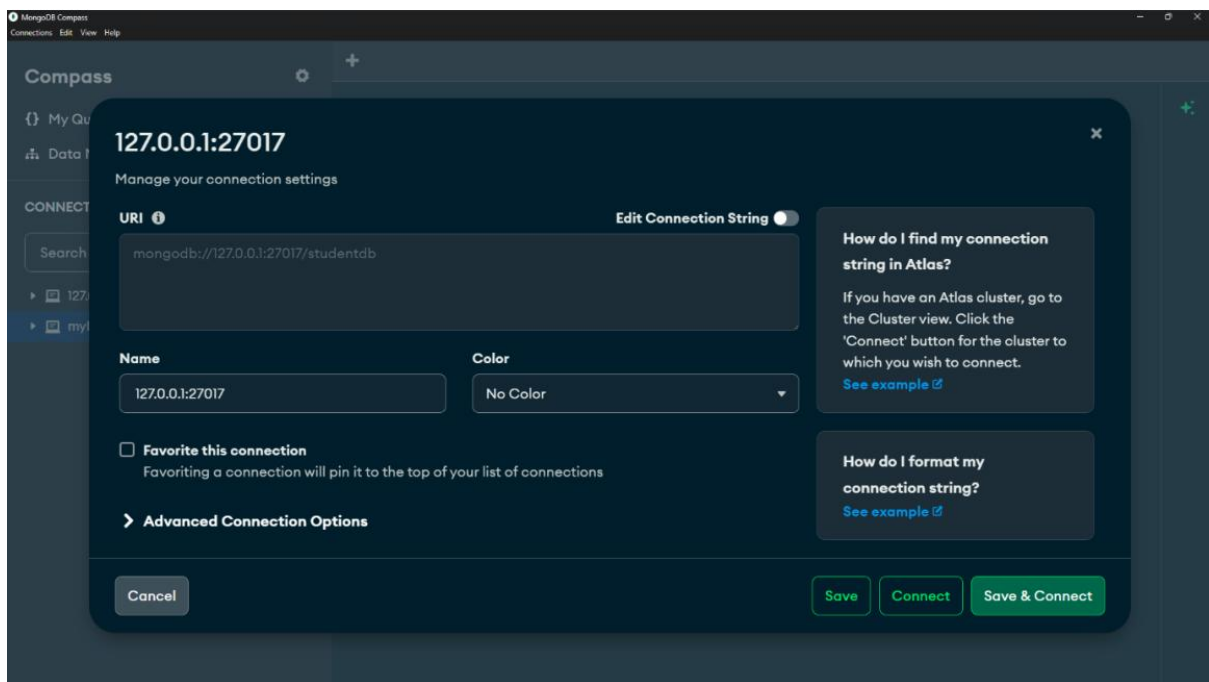
Database Creation: A database named **studentdb** was created using MongoDB Compass to store student records.

Inside the database, a collection named **students** is created.

MongoDB stores student records as documents in JSON format.

EXAMPLE:

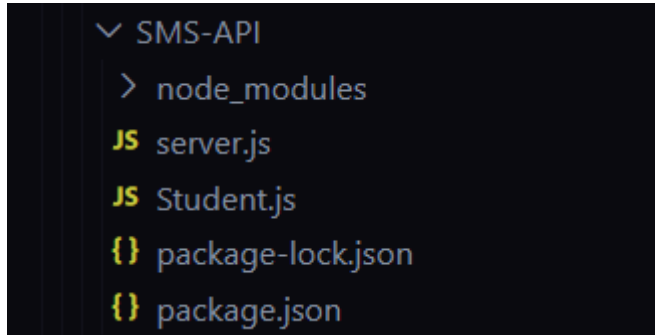
```
{
  name: "Vishesh Gurnani",
  age: 19,
  course: "BCA"
}
```



```
_id: ObjectId('69b23d6578ae8854028b5d4a')
name : "Vishesh Gurnani"
age : 19
course : "BCA"
__v : 0
```

Project Structure

The project folder structure in Visual Studio Code is organized as follows:



Here,

- server.js – Contains the API logic and server configuration.
- Student.js – Defines the MongoDB schema and model.
- package.json – Stores project metadata and dependencies.
- node_modules – Contains installed libraries.

Student Schema

Student.js

```
const mongoose = require("mongoose");

const StudentSchema = new mongoose.Schema({

  name: String,

  age: Number,

  course: String

});

module.exports = mongoose.model("Student", StudentSchema);
```

Here,

- name – Stores the student's name.
- age – Stores the student's age.
- course – Stores the course the student is enrolled in

Server Implementation

The server file connects to MongoDB and defines API routes.

Server.js

```
const express = require("express");
const mongoose = require("mongoose");
const Student = require("./Student");
const app = express();
app.use(express.json());

//MONGO DB CONNECTION
mongoose.connect("mongodb://127.0.0.1:27017/studentdb")
.then(()=>console.log("MongoDB Connected"))
.catch(err=>console.log(err));

// Add Student (POST method)
app.post("/students", async (req, res) => {
  const student = new Student(req.body);
  const result = await student.save();
  res.send(result);
});

// Get Students (GET method)
app.get("/students", async (req, res) => {
  const students = await Student.find();
  res.send(students);
});
```

```
// Update Student (PUT method)

app.put("/students/:id", async (req, res) => {

  const student = await Student.findByIdAndUpdate(

    req.params.id,

    req.body,

    { new: true }

  );

  res.send(student);

});


// Delete Student (DELETE method)

app.delete("/students/:id", async (req, res) => {

  const student = await Student.findByIdAndDelete(req.params.id);

  res.send(student);

});


// CREATING LOCAL HOST SERVER

app.listen(3000,()=>{

  console.log("http://localhost:3000/")

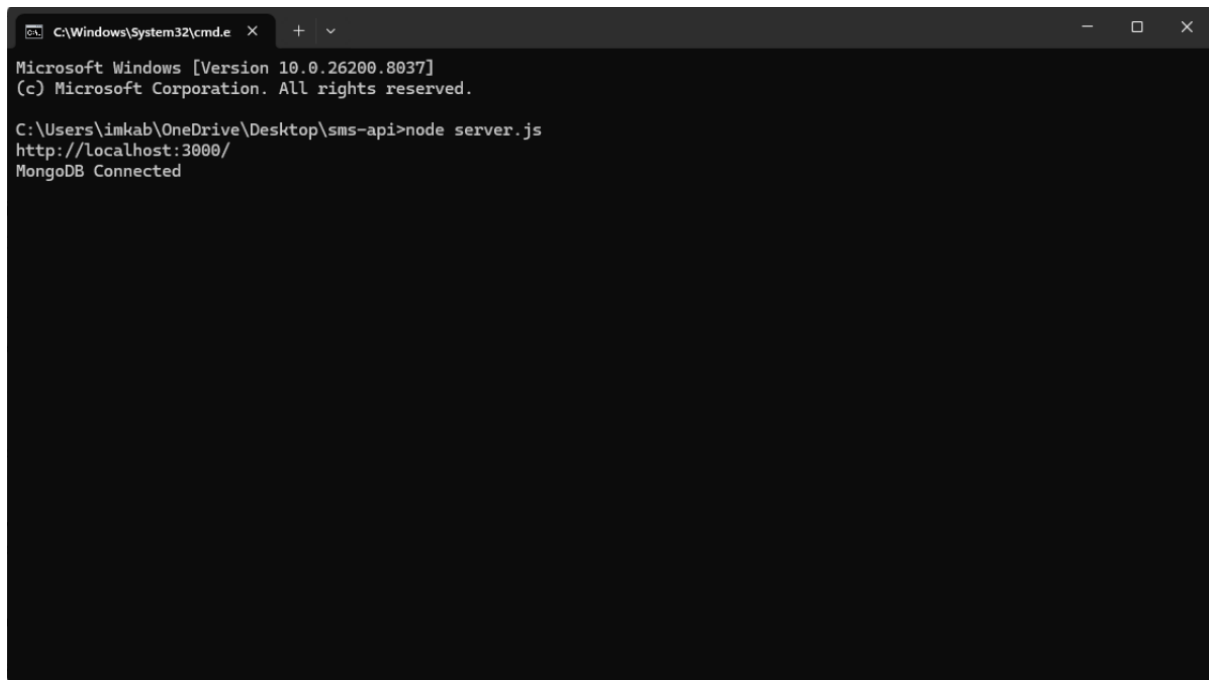
});
```

Running the Server in Terminal

The server is started using the following command:

```
node server.js
```

Once executed successfully, the terminal displays:



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\imkab\OneDrive\Desktop\sms-api>node server.js
http://localhost:3000/
MongoDB Connected
```

API Testing using Postman

Postman is used to test the API endpoints.

- POST Request:

Endpoint:

POST <http://localhost:3000/students>

Body (JSON Format):

```
{
  "name": "Vishesh",
  "age": 19,
  "course": "BCA"
}
```

- GET Request:

Endpoint:

GET <http://localhost:3000/students>

This request retrieves all student records stored in MongoDB.

- PUT Request:

Endpoint:

PUT <http://localhost:3000/students/:id>

EXAMPLE:

```
{  
  "name": "Vishesh Gurnani",  
  "age": 19,  
  "course": "BCA"  
}
```

- DELETE Request:

Endpoint:

DELETE <http://localhost:3000/students/:id>

Browser Output:

When the API endpoint is accessed in the browser:

<http://localhost:3000/students>

The browser displays all student records stored in the MongoDB database in JSON format.



```
[  
  {  
    "_id": "69b23d6578ae8854028b5d4a",  
    "name": "Vishesh Gurnani",  
    "age": 19,  
    "course": "BCA",  
    "__v": 0  
  },  
  {  
    "_id": "69b23d6e78ae8854028b5d4c",  
    "name": "Soham Sen",  
    "age": 19,  
    "course": "BCA",  
    "__v": 0  
  },  
  {  
    "_id": "69b23d7d78ae8854028b5d4e",  
    "name": "Saurabh Sen",  
    "age": 24,  
    "course": "BCA",  
    "__v": 0  
  },  
]
```

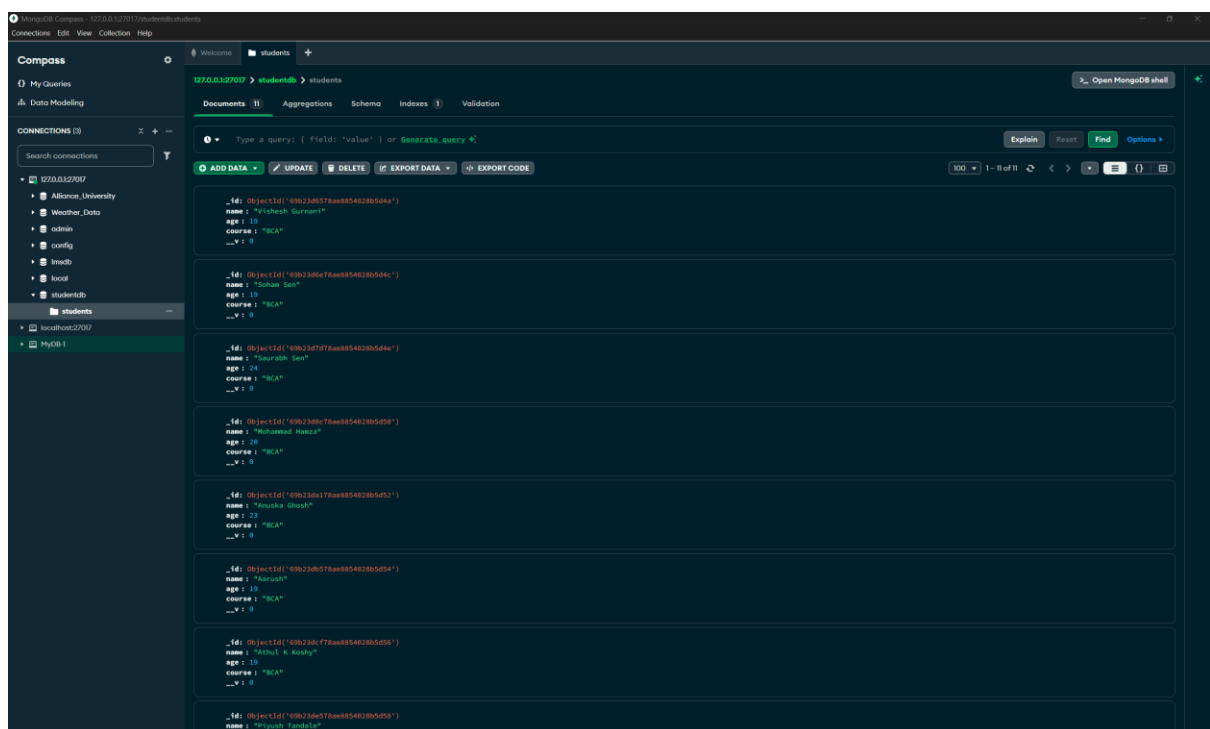
MongoDB Compass Database View

MongoDB Compass allows users to visually inspect the database.

In this project:

- Database: studentdb
- Collection: students
- Documents contain student name, age, and course details.

Compass displays the data in JSON format for easy analysis.



Results:

The Student Management System API was successfully implemented using Node.js and MongoDB.

The application allows users to:

- Add new students to the database.
- Retrieve student records.
- Update student information.
- Delete student records.

All API endpoints were successfully tested using Postman, and the data was correctly stored and displayed in MongoDB Compass.

Conclusion

This project successfully demonstrated how a REST API can be created using **Node.js and Express** and connected to a **MongoDB database**. By integrating **VS Code for development, MongoDB Compass for database management, and Postman for API testing**, a functional student management API was built. The system supports CRUD operations, allowing users to add, retrieve, update, and delete student records through HTTP requests. This project provided practical experience in backend development, database connectivity, and API testing, forming a strong foundation for building more advanced web applications.